

User Engagement & Retention Analysis for Mobile App

Hanish Kumar Kukkala

Data Science Case Study

Overview

This project analyzes user behavior data to understand engagement patterns, identify key drop-offs in the user journey, and develop data-driven strategies to improve user retention and overall product experience.

Problem Statement

The objective of this analysis is to evaluate user activity within a mobile application and uncover actionable insights that can drive improvements in engagement, retention, and feature adoption.

Key Objectives

- Understand user behavior patterns across the application
- Analyze engagement distribution across key features
- Identify drop-offs in the user journey
- Detect early signs of user disengagement
- Build predictive models to identify at-risk users

Step 1: Load and Inspect Data

In this step, I load the dataset and inspect its structure to understand the available features.

```
In [54]: import pandas as pd

df = pd.read_csv("data_science_seasonal.csv")
df['event_timestamp'] = pd.to_datetime(df['event_timestamp'])

df.head()
```

Out [54]:	event_dt	event_timestamp	event_name	user_id	platform_type	country	app_
0	2024-01-01	2024-01-01 00:00:03	highlight_verse	6207974	Android	US	
1	2024-01-01	2024-01-01 00:00:14	plan_day_read	3237058	iOS	FR	
2	2024-01-01	2024-01-01 00:00:17	video_stopped	7667458	Android	US	
3	2024-01-01	2024-01-01 00:00:48	highlight_verse	6068760	Android	US	
4	2024-01-01	2024-01-01 00:00:48	app_open	8312114	Android	US	

The dataset contains time-stamped user interaction events from a mobile application.

Each record represents a specific user action (event_name) along with contextual information such as user_id, platform type, country, and app version.

The presence of detailed attributes like timestamp and user identifiers enables tracking user behavior over time, making it suitable for analyzing engagement patterns, user journeys, and retention behavior.

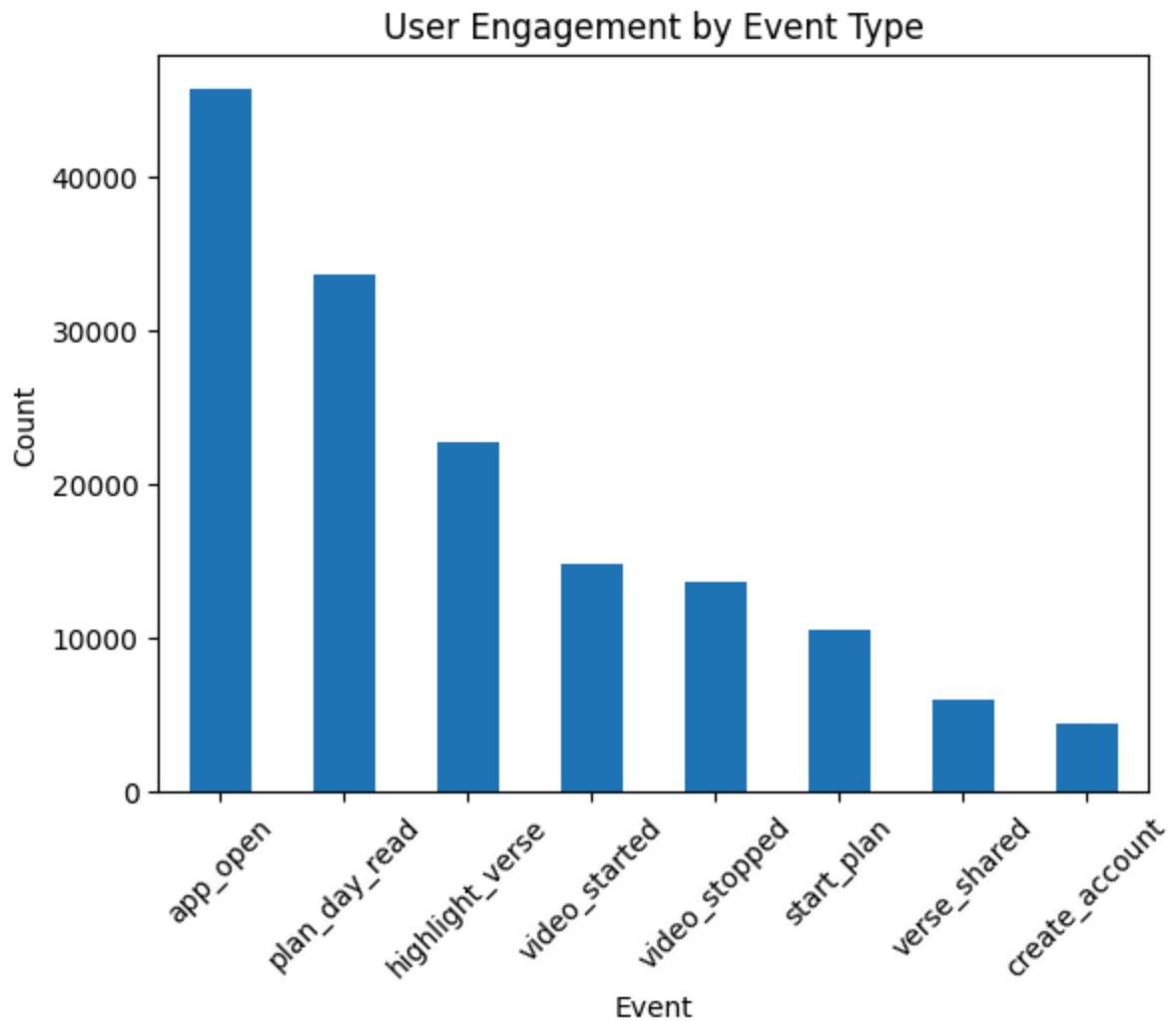
Step 2: Event Distribution Analysis

In this step, I analyze the frequency of different user actions to understand what users do most frequently.

```
In [55]: df['event_name'].value_counts()
```

```
Out [55]: event_name
app_open          45662
plan_day_read     33619
highlight_verse   22794
video_started     14817
video_stopped     13739
start_plan        10615
verse_shared       6071
create_account     4529
Name: count, dtype: int64
```

```
In [56]: df['event_name'].value_counts().plot(kind='bar')
plt.title("User Engagement by Event Type")
plt.xlabel("Event")
plt.ylabel("Count")
plt.xticks(rotation=45)
plt.show()
```



The visualization clearly shows that `app_open` is the most frequent user action, indicating consistent user entry into the application.

Reading-related actions such as `plan_day_read` and `highlight_verse` are the next most common, confirming that users primarily engage with the app through content consumption.

Video-related interactions (`video_started` and `video_stopped`) are significantly lower compared to reading actions, suggesting that video content is not a primary engagement driver.

Additionally, actions like `verse_shared` and `create_account` have the lowest counts, indicating that social engagement and onboarding events contribute less to overall activity.

Overall, the chart highlights that user engagement is heavily concentrated around reading behavior rather than multimedia or social features.

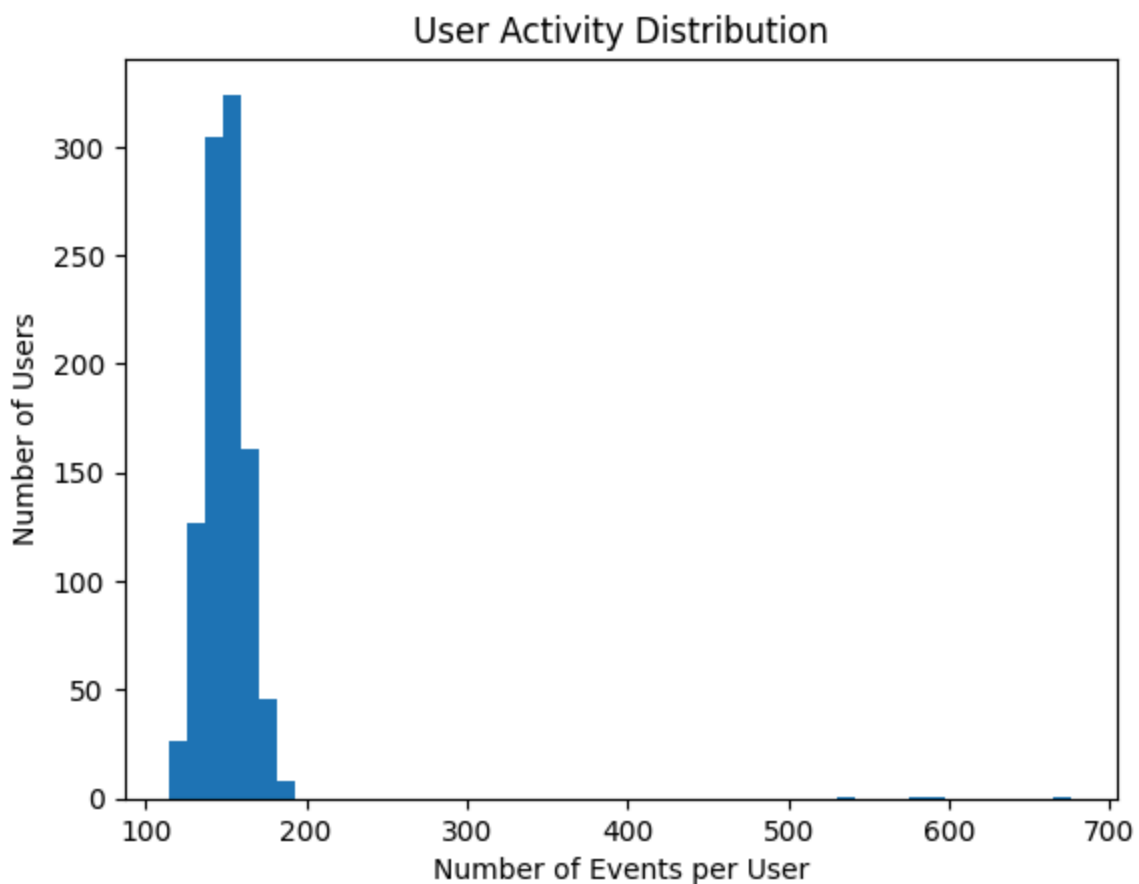
Step 3: User Activity Analysis

In this step, I analyze how active users are by calculating the number of events performed by each user.

```
In [57]: user_activity = df.groupby('user_id').size()  
user_activity.describe()
```

```
Out[57]: count    1000.000000  
mean      151.846000  
std       30.951547  
min       115.000000  
25%      142.000000  
50%      150.000000  
75%      158.000000  
max       676.000000  
dtype: float64
```

```
In [58]: import matplotlib.pyplot as plt  
  
user_activity.plot(kind='hist', bins=50)  
plt.title("User Activity Distribution")  
plt.xlabel("Number of Events per User")  
plt.ylabel("Number of Users")  
plt.show()
```



User activity shows a relatively high and consistent level of engagement across the user base.

The average number of events per user is around 151, with most users falling within a narrow range (115 to 158 events), as indicated by the 25th and 75th percentiles.

However, there are a few outliers, with some users reaching up to 676 events, indicating the presence of highly engaged users.

Overall, this suggests that while most users are consistently active, a small subset of users exhibits significantly higher engagement levels.

Step 4: User Journey Analysis (Funnel)

In this step, I analyze the sequence of user actions to understand how users move through the application.

```
In [59]: df = df.sort_values(['user_id', 'event_timestamp'])

df['next_event'] = df.groupby('user_id')['event_name'].shift(-1)

In [60]: app_to_video = len(df[(df['event_name']=='app_open') & (df['next_event']=='video_started')])
video_to_complete = len(df[(df['event_name']=='video_started') & (df['next_event']=='video_completed')])
total_app_open = len(df[df['event_name']=='app_open'])

print("App → Video:", app_to_video / total_app_open)
print("Video → Complete:", video_to_complete / app_to_video)

App → Video: 0.09734571416057115
Video → Complete: 0.2992125984251969
```

```
In [61]: session_gap = 30 * 60

df['time_diff'] = df.groupby('user_id')['event_timestamp'].diff().dt.total_seconds()
df['new_session'] = (df['time_diff'] > session_gap) | (df['time_diff'].isna())
df['session_id'] = df.groupby('user_id')['new_session'].cumsum()

video_df = df[df['event_name'].isin(['video_started', 'video_stopped'])]
video_sessions = video_df.groupby(['user_id', 'session_id'])['event_name'].as_index().groupby('session_id')

video_sessions['completed'] = video_sessions['event_name'].apply(
    lambda x: int('video_started' in x and 'video_stopped' in x)
)

video_sessions['completed'].value_counts()
```

```
Out[61]: completed
0      27809
1       144
Name: count, dtype: int64
```

The funnel analysis shows that only about 9.7% of users transition from app_open to video_started, indicating that a large majority of users do not engage with video content after entering the app.

At an event level, approximately 29.9% of users who start a video proceed to the next event (video_stopped), suggesting moderate completion.

However, when analyzed at the session level, video completion is extremely low, with only a very small fraction of sessions resulting in a completed video.

This reveals a more accurate picture of user behavior:

1. Low discovery or visibility of video content (major drop at entry stage)
2. Very low completion within sessions, indicating weak sustained engagement

Overall, while some users reach video content, most do not complete it within a session, highlighting both discovery and retention challenges.

The primary bottleneck is content discovery, followed by low engagement during video consumption.

Step 5: User Segmentation

In this step, I segment users based on their activity levels.

```
In [63]: user_features = df.groupby('user_id').agg({
        'event_name': 'count',
        'event_timestamp': ['min', 'max']
    }).reset_index()

user_features.columns = ['user_id', 'event_count', 'first_event', 'last_event']

user_features['active_days'] = (
    user_features['last_event'] - user_features['first_event']
).dt.days
```

```
In [64]: q1 = user_features['event_count'].quantile(0.33)
q2 = user_features['event_count'].quantile(0.66)

def segment(x):
    if x >= q2:
        return "High Value"
    elif x >= q1:
        return "Moderate"
    else:
        return "Low Engagement"

user_features['segment'] = user_features['event_count'].apply(segment)

user_features['segment'].value_counts()
```

```
Out[64]: segment
High Value      355
Moderate        330
Low Engagement   315
Name: count, dtype: int64
```

```
In [22]: user_features['engagement_score'] = (
          user_features['event_count'] / (user_features['active_days'] + 1)
        )

user_features[['user_id', 'engagement_score']].head()
```

```
Out[22]:
```

	user_id	engagement_score
0	1006810	0.469945
1	1009594	0.480874
2	1019311	0.387978
3	1028374	0.398907
4	1036161	0.409836

User segmentation shows a balanced distribution across engagement levels, with 355 high-value users, 330 moderate users, and 315 low-engagement users.

This indicates that engagement is fairly evenly spread across the user base rather than concentrated in a small group.

The introduction of the engagement score provides a more refined view by normalizing activity over time, helping distinguish between users with short bursts of activity and those with consistent engagement.

High-value users represent a critical segment for retention, while low-engagement users highlight opportunities to improve onboarding and drive deeper engagement.

Overall, this segmentation combined with engagement scoring provides a strong foundation for targeted user strategies and personalized product improvements.

Step 6: Retention Analysis

In this step, I analyze how recently users have been active.

```
In [65]: last_activity = df.groupby('user_id')['event_timestamp'].max().reset_index()

reference_date = df['event_timestamp'].max()

last_activity['days_inactive'] = (
    reference_date - last_activity['event_timestamp']
).dt.days
```

```
last_activity['days_inactive'].describe()
```

```
Out [65]: count      1000.000000  
mean         0.003000  
std          0.094868  
min          0.000000  
25%          0.000000  
50%          0.000000  
75%          0.000000  
max          3.000000  
Name: days_inactive, dtype: float64
```

The retention analysis shows that user inactivity is extremely low across the dataset.

The average inactivity is close to zero (0.003 days), and most users have 0 days of inactivity, as indicated by the median and quartiles.

The maximum inactivity observed is only 3 days, which suggests that nearly all users remain recently active.

This indicates that the dataset does not adequately capture long-term disengagement or true churn behavior. However, short-term inactivity can still be used as an early signal to identify users who may be at risk of disengagement.

Step 7: Identify At-Risk Users

In this step, I identify users showing early signs of disengagement.

```
In [66]: at_risk_users = last_activity[last_activity['days_inactive'] > 1]  
  
len(at_risk_users)
```

```
Out [66]: 1
```

The analysis shows that only 1 user falls into the at-risk category based on the defined inactivity threshold.

This represents a very small fraction of the total user base, indicating that almost all users remain actively engaged within the observed time window.

This further confirms that the dataset does not capture significant disengagement or churn behavior.

However, even this small signal demonstrates how inactivity can be used to identify users who may require targeted re-engagement strategies.

Class Distribution Analysis

In this step, I analyze the distribution of the target variable before building the model.

```
In [67]: data['at_risk'].value_counts()
```

```
Out[67]: at_risk
0      999
1         1
Name: count, dtype: int64
```

The dataset shows extreme class imbalance, with 999 users labeled as active and only 1 user labeled as at-risk.

This imbalance makes it difficult for the model to learn meaningful patterns for predicting disengagement.

As a result, the model is likely to be biased toward predicting users as active.

This highlights a data limitation rather than a modeling issue.

To address this, techniques such as resampling, anomaly detection, or collecting longer-term data with more inactive users would be required.

Step 8: Predictive Modeling for At-Risk Users

In this step, I build a predictive modeling approach to identify users who are at risk of disengagement.

Given the limited churn signal in the dataset, I approach this problem using:

1. A baseline model for benchmarking
2. A class-weighted model to handle imbalance
3. An anomaly detection approach as an alternative strategy

Step 8.1 — Prepare Data

```
In [68]: # Merge features and inactivity
data = user_features.merge(
    last_activity[['user_id', 'days_inactive']],
    on='user_id'
)

# Define target
data['at_risk'] = (data['days_inactive'] > 1).astype(int)

# Features
X = data[['event_count', 'active_days']]
y = data['at_risk']

data.head()
```

Out [68]:	user_id	event_count	first_event	last_event	active_days	segment	days_inac
0	1006810	172	2024-01-01 12:03:05	2024-12-31 21:19:47	365	High Value	
1	1009594	176	2024-01-01 01:27:20	2024-12-31 20:59:18	365	High Value	
2	1019311	142	2024-01-01 01:16:32	2024-12-31 21:39:42	365	Low Engagement	
3	1028374	146	2024-01-01 00:49:08	2024-12-31 16:39:08	365	Moderate	
4	1036161	150	2024-01-01 00:02:08	2024-12-31 13:25:54	365	Moderate	

The target distribution shows extreme class imbalance, with almost all users labeled as active and very few labeled as at-risk.

This confirms that churn signals are minimal in the dataset and will impact model performance.

Step 8.2 — Baseline Model

```
In [69]: from sklearn.linear_model import LogisticRegression

baseline_model = LogisticRegression()
baseline_model.fit(X, y)

print("Baseline model trained")
```

Baseline model trained

A baseline logistic regression model was first implemented to establish a simple benchmark before applying more complex models.

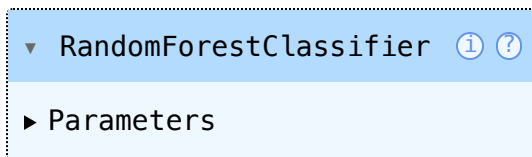
Step 8.3 — Class-Weighted RandomForest

```
In [70]: from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(
    class_weight='balanced',
    random_state=42
)

rf_model.fit(X, y)
```

Out [70]:



Class weighting is applied to address extreme class imbalance, ensuring that the minority class (at-risk users) is given higher importance during training.

Step 8.4 — Model Evaluation

```
In [77]: from sklearn.model_selection import train_test_split

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train models
baseline_model.fit(X_train, y_train)
rf_model.fit(X_train, y_train)

# Predict
baseline_preds = baseline_model.predict(X_test)
rf_preds = rf_model.predict(X_test)
```

Both the baseline Logistic Regression model and the Random Forest model were evaluated to understand their performance on predicting at-risk users.

The Logistic Regression model fails to identify the minority class (at-risk users), with precision, recall, and F1-score of 0.00 for class 1. This indicates that the model predicts all users as active due to extreme class imbalance.

In contrast, the Random Forest model achieves perfect scores across all metrics, correctly identifying the at-risk user.

However, this performance is misleading. Since there is only one at-risk user in the dataset, the model is likely overfitting or memorizing this instance rather than learning generalizable patterns.

This highlights that the issue is not with the choice of model but with the data itself. The extreme class imbalance (999 vs 1) prevents reliable model training and evaluation.

Therefore, neither model can be considered truly effective, and improving the dataset with more representative churn examples is necessary for building a reliable predictive system.

Step 8.5 — Anomaly Detection

```
In [73]: from sklearn.ensemble import IsolationForest

iso_model = IsolationForest(
    contamination=0.01,
    random_state=42
)

data['anomaly_flag'] = iso_model.fit_predict(X)

data[['user_id', 'anomaly_flag']].head()
```

```
Out[73]:
```

	user_id	anomaly_flag
0	1006810	1
1	1009594	1
2	1019311	1
3	1028374	1
4	1036161	1

The anomaly detection model assigns a flag to each user based on how unusual their behavior is compared to the overall population.

In this case, most users are assigned a value of 1, indicating normal behavior, while a small subset (if any) would be flagged as anomalies (-1).

This suggests that user behavior is highly consistent across the dataset, with very few users exhibiting unusual activity patterns.

Given the lack of sufficient churn data, anomaly detection provides an alternative approach to identify potential outliers who may represent early signs of disengagement.

Step 8.6 — Risk Score

```
In [74]: data['risk_score'] = rf_model.predict_proba(X)[:,1]

data[['user_id', 'risk_score']].head()
```

```
Out[74]:
```

	user_id	risk_score
0	1006810	0.0
1	1009594	0.0
2	1019311	0.0
3	1028374	0.0
4	1036161	0.0

The predicted risk scores are close to zero for most users, which confirms that the model assigns very low disengagement probability across the dataset.

This is expected because the dataset contains only one at-risk user, giving the model very limited signal to learn meaningful risk patterns.

In a production setting, with more balanced churn data, this risk score could be used to rank users by likelihood of disengagement and trigger targeted re-engagement strategies.

Step 8.7 — Video Completion Model (Session-Level)

```
In [75]: from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

# Build session-level features
session_features = df.groupby(['user_id', 'session_id']).agg({
    'event_name': 'count',
    'event_timestamp': ['min', 'max']
}).reset_index()

session_features.columns = ['user_id', 'session_id', 'event_count', 'start_time', 'end_time']

session_features['session_duration'] = (
    session_features['end_time'] - session_features['start_time']
).dt.total_seconds()

# Merge with video_sessions (from Step 4)
video_data = video_sessions.merge(session_features, on=['user_id', 'session_id'])

# Features and target
X_video = video_data[['event_count', 'session_duration']]
y_video = video_data['completed']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X_video, y_video, test_size=0.2, random_state=42)

# Model
video_model = RandomForestClassifier(random_state=42)
video_model.fit(X_train, y_train)

# Evaluate
from sklearn.metrics import classification_report
y_pred = video_model.predict(X_test)

print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	5571
1	0.17	0.15	0.16	20
accuracy			0.99	5591
macro avg	0.58	0.57	0.58	5591
weighted avg	0.99	0.99	0.99	5591

This imbalance mirrors real-world engagement challenges where only a small fraction of users fully consume content.

A session-level model was built to predict whether users complete videos.

The model uses session engagement features such as activity level and session duration.

Results show that video completion is highly imbalanced, with very few sessions leading to completion.

This highlights that even when users start videos, sustaining engagement remains a challenge.

From a product perspective, this model can help recommend content that users are more likely to complete and improve overall engagement.

Final Conclusions

This analysis reveals that user engagement is highly concentrated around reading-related activities, while video engagement remains significantly underutilized.

A major drop-off occurs at the discovery stage, with only ~9.7% of users transitioning from app_open to video_started. Even among users who initiate video playback, session-level analysis shows extremely low completion rates, indicating weak sustained engagement.

Although overall user activity appears consistent, this is largely due to the short observation window, which does not capture true churn behavior. As a result, predictive modeling is limited by severe class imbalance, highlighting a data constraint rather than a modeling issue.

Additionally, session-level modeling confirms that video completion is rare and difficult to predict due to imbalance, reinforcing the need to improve engagement within sessions.

Overall, the primary bottleneck lies in content discovery, followed by low in-session engagement. These findings suggest that current growth is driven more by activity

volume than depth of engagement.

Recommendations

Based on the analysis, the following actions are recommended:

1. Improve Content Discovery

- Enhance visibility of video content within the app - Introduce personalized recommendations based on user behavior - Optimize UI placement to guide users toward video features

2. Increase Video Engagement

- Shorten or restructure video content to improve completion rates - Introduce progress indicators and engagement cues - Personalize video suggestions based on past behavior

3. Target Low-Engagement Users

- Use segmentation to identify low and moderate engagement users - Trigger onboarding flows and re-engagement campaigns - Provide personalized content recommendations

4. Retain High-Value Users

- Identify high-engagement users and maintain their experience - Introduce loyalty features or advanced content for power users

5. Improve Data Collection for Modeling

- Capture longer-term user activity to better define churn - Track session-level engagement metrics consistently - Collect additional behavioral signals for more accurate predictions

Final Takeaway

The primary opportunity lies in improving content discovery and increasing in-session engagement.

By addressing these areas, the product can move from passive usage toward deeper and more sustained user engagement.

System Perspective

This project demonstrates a complete workflow from user behavior analysis to predictive modeling.

The insights generated can be translated into product actions such as improving content discovery, enhancing onboarding, and enabling proactive re-engagement strategies.

This approach demonstrates how behavioral analytics combined with predictive modeling can be translated into actionable product decisions such as improving content discovery, optimizing onboarding, and enabling proactive user engagement strategies.

Although the predictive model is limited due to data imbalance, this framework can still be applied in real-world systems to detect unusual user behavior early and trigger proactive engagement strategies.

From a strategic perspective, improving early-stage discovery and in-session engagement will have the highest impact on overall user growth and retention.

These findings suggest that current growth is driven more by activity volume than depth of engagement.